Jia *et al. Cybersecurity* (2026) 9:245
https://doi.org/10.1186/s42400-026-00683-3

Cybersecurity

RESEARCH Open Access

A dynamic malware detection method based on multi-modal features

Lexin Jia[1,3,4], Jian Zhang[1,2,3,4*], Xiangyi Wang[1,2,3,4] and Wei Zheng[5]

Abstract

The rapid increase of sophisticated malware poses significant challenges to existing detection methods. To address the challenge, we propose a dynamic malware detection method based on multi-modal features. This approach utilizes two distinct modalities from in-band and out-of-band: the API (Application Programming Interface) sequence features obtained in sandbox, and the image features converted by memory dump files obtained out of VM (Virtual Machine). A multi-modal deep learning model is proposed to fuse these features to distinguish sophisticated malware. Experiments show that our method is effective, achieving 98.6% accuracy on a dataset collected from VirusTotal and CNET. It demonstrates significant improvements over single-modal methods using either text or image features alone, while combining the advantages of out-of-band and in-band dynamic detection, providing an effective solution for malware detection.

Keywords Malware, Dynamic detection, API sequence, Memory, Multi-modal fusion

Introduction

Malware refers to software designed with malicious intent, containing features or capabilities that can potentially cause harm directly or indirectly to the user and the user's computer system (International Organization for Standardization 2023). The rapid increase in the quantity and diversity of malware has posed a persistent threat. According to the AV-TEST Institute report, more than 450,000 new malware samples were registered every day in 2024 (AV-TEST Institute 2024). There is an urgent need to improve the capability of accurately detecting sophisticated malware.

Depending on the need of executing samples, malware detection methods can be categorized into static and dynamic methods (Sihwail et al. 2018; Aboaoja et al. 2022). Static detection methods do not require executing malware and can provide rapid detection results (Yamany et al. 2024). However, static analysis struggles to accurately detect malware that employs packing and encryption techniques. They can change their static features or use obfuscation techniques to mask their content to avoid detection (Brosolo et al. 2026). In contrast, dynamic detection methods execute samples in the monitored environment (Or-Meir et al. 2019), which can catch runtime software behaviors to identify malicious activities (Kim et al. 2017). Thus, dynamic detection methods are less susceptible to malware evasion methods.

API call sequence feature is one of the most widely used feature in dynamic detection. They are usually obtained from a sandbox and can effectively reflect the runtime behaviors of a program (Zhang et al. 2025; Amer and Zelinka 2020). However, API call sequence features are categorized as in-band features. It shares the same observation perspective as the operating system, making it vulnerable to evasion techniques. Some malware

*Correspondence:
Jian Zhang
zhang.jian@nankai.edu.cn
[1] College of Cryptology and Cyber Science, Nankai University, Tongyan Road 38, Tianjin 300350, China
[2] College of Computer Science, Nankai University, Tongyan Road 38, Tianjin 300350, China
[3] Tianjin Key Laboratory of Network and Data Security Technology, Tongyan Road 38, Tianjin 300350, China
[4] Key Laboratory of Data and Intelligent System Security Ministry of Education, Tongyan Road 38, Tianjin 300350, China
[5] Tianjin Institute of Navigation Instrument, Tianjin, China

© The Author(s) 2026. **Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

may bypass API calls and directly manipulate memory or exploit system vulnerabilities, which can reduce the effectiveness of API-based detection (Cui et al. 2024). Various memory-based malware detection methods take memory snapshots of processes or the entire memory (Patil et al. 2020; Darem et al. 2021) outside the guest operating system. This allows them to obtain more trustworthy and tamper-resistant information compared to in-band features. VMs are widely used in these methods because they provide an isolation from the host and support rapid creation and destruction for detection (Zheng and Zhang 2022). Nevertheless, these memory snapshots only reflect the state at a specific moment and cannot capture all dynamically changing information of the sample (Dehfouli and Lashkari 2025). In addition, traditional Virtual Machine Introspection (VMI) techniques often require substantial semantic reconstruction to interpret raw memory data, introducing significant complexity and overhead. This further highlights the importance of dimensionality reduction.

Both in-band and out-of-band features used in isolation exhibit limitations in their observation scopes, resulting in an incomplete representation of malicious behaviors. Sophisticated malware may use evasion techniques to avoid specific detection methods that rely on single-modal features. Consequently, effectively combining multiple data sources and leveraging complementary features has become an important direction in malware detection research (Jeon et al. 2024). Multi-modal deep learning technologies have emerged in malware detection. By fusing multiple features such as images and text, detection methods can more effectively analyze the diverse characteristics of malware (Zhang et al. 2024).

To enhance the ability to detect sophisticated malware, we propose a multi-modal approach that integrates API call sequence text features and image features that are converted from memory dump files. We run samples in the Cuckoo sandbox and gather the API call sequences. Then, we use an improved Long Short-Term Memory (LSTM) network to extract text features. Simultaneously, we run samples in a KVM (Kernel-based Virtual Machine), and then dump the entire memory out of the VM. Unlike traditional methods, we select a key part of the memory file and convert it into a color image. The image features are extracted through an enhanced Convolutional Neural Network (CNN). The image features and text features are then fused a multi-modal feature representation for detection. Our method addresses the limitations of single-modal detection and combines the advantages of out-of-band and in-band methods, enhancing the ability to detect sophisticated malware.

The main contribution of this paper can be summarized as follows:

- We propose a new feature extraction method to process API call sequences obtained through in-band analysis. In addition, we extract key memory from the virtual machine out-of-band and convert them into color images. These complementary approaches significantly enhance malware analysis. By providing deeper insights into both in-band and out-of-band behaviors, they enable more effective malware detection.
- We employ a fine-tuned LSTM for modeling API sequence features and a fine-tuned CNN for image-based feature extraction. The extracted features are subsequently fused to enable multi-modal malware detection. By leveraging complementary dynamic characteristics from different modalities, the proposed approach enhances the overall detection capability.

Related work

API-based detection

API call sequences can represent the runtime behavior of a program (Wei et al. 2026). Malware often exploits APIs provided by applications or operating systems to perform malicious operations (Syeda and Asghar 2024). Consequently, API call sequences are widely used in current malware classification and detection techniques. Specifically, Gibert et al. exploited API call sequences to obtain features and used machine learning techniques for malware detection (Gibert et al. 2020). Furthermore, various deep learning methods, such as CNN and LSTM networks, have been utilized to analyze API call sequences for malware detection (Bensaoud and Kalita 2024; Karat et al. 2024). For example, Tobiyama et al. employed a Recurrent Neural Network (RNN) model to extract features from API sequences, which were then used to train a CNN to classify feature images (Tobiyama et al. 2016). Lu et al. applied a bidirectional residual network to categorize API call sequences while eliminating redundant information (Lu et al. 2019). Li et al. utilized Bi-LSTM to extract and integrate inherent features of API sequences (Li et al. 2022).

API call sequence-based malware detection has a relatively simple feature extraction process and is able to capture the runtime behavior of malware. However, some malware directly manipulate memory or exploit system vulnerabilities, thereby evading API-based detection (Zhang et al. 2022). Meanwhile, malicious and benign software may exhibit similar API call sequences, increasing the risk of false positives (Wu et al. 2025). Additionally, anti-sandbox malware can detect sandbox environments and evade or delay malicious actions, avoiding suspicious API calls (Cui et al. 2023).

Memory-based detection

A VM is a virtualized environment created on a physical host (Rozehkhani et al. 2024). It supports fast reconstruction and multiple operating systems, making it suitable for modern memory forensics (Zhang et al. 2023). Based on the positioning of the data collection tools, VM-based detection can be classified into in-VM detection and out-of-VM detection (Zheng et al. 2023). In-VM detection supports real-time monitoring of malicious behaviours within the guest environment (He and Li 2024). However, in-VM detection relies on the guest operating system's internal mechanisms. It may therefore fail to detect hidden processes created by sophisticated malware. In contrast, out-of-VM detection utilizes external tools to inspect the target system from the hypervisor level (Mishra et al. 2021), offering a more transparent and tamper-resistant perspective.

Memory is a volatile storage medium used by computers to temporarily hold data and the operating system during execution (Gbedawo et al. 2023). Memory forensics involves capturing and examining a computer system's memory to gather information about its operational state (Shree et al. 2022). In memory forensics, some memory forensic files are represented and processed as images (Maniriho et al. 2024). Malware can be identified by analyzing memory forensic files represented in the form of images (Arfeen et al. 2022). In particular, Yücel et al. introduced a novel approach to imaging malware using artifacts from memory operations and demonstrated how image comparison techniques can be applied for detection (Yücel and Koltuksuz 2020). Wang et al. proposed the MDMV framework, which analyzes malware by extracting runtime artifacts from KVM physical memory dumps. The method transforms memory data into compact visual representations using SimHash and applies an attention-based neural architecture to capture structural patterns (Wang et al. 2024).

However, memory images capture only a snapshot at a particular moment, which may fail to record all dynamically changing information and transient malicious activities (Carrier 2021). Moreover, some existing techniques convert memory data covering the entire memory space of a system into a binary memory image (Dener et al. 2022). Using the entire memory for analysis requires many resources, making dimensionality reduction important.

Multi-modal detection

To overcome the limitations of single-modal data, researchers have proposed multi-modal fusion techniques and applied them across a range of tasks (Kim et al. 2018; Wang et al. 2025). In multi-modal frameworks, models are typically designed and trained to simultaneously process multiple types of data (e.g., images, text, and sequences) to learn more comprehensive feature representations (Liu et al. 2018; Qin et al. 2025). Multi-modal fusion techniques have also been applied to malware detection (Almashhadani et al. 2022; Yi et al. 2025). Chen et al. proposed an Android malware detection method that combines structural and semantic features using a Class Set Call Graph (CSCG) and extracts features using a Graph Attention Network (GAT). They then used a deep multi-modal network to fuse these features with permission data, thereby improving detection performance (Chen et al. 2024). Li et al. utilized a feature-based multi-modal fusion approach during the feature extraction stage, which significantly improved classification performance on a highly imbalanced dataset of malware families (Li et al. 2022).

Gibert et al. introduced HYDRA, which effectively achieved malware classification by learning various static feature types, including API function calls, assembly mnemonics, and raw byte sequences. Although HYDRA effectively exploits complementary branches via late feature concatenation, this fusion strategy is inherently tailored for homogeneous features derived from the same underlying static file. It lacks the mechanism to coordinate heterogeneous modalities with extreme spatiotemporal discrepancies. Furthermore, it relies on static analysis, which is vulnerable to sophisticated obfuscation techniques and fails to capture dynamic runtime behaviors (Gibert et al. 2020).

Zheng et al. proposed a multimodal-based abnormal behavior detection method in a virtualization environment. Specifically, it collects virtual hardware features at the virtualization layer, as well as real hardware and system call features at the physical machine level. Next, it extracts memory features from runtime memory snapshots. The hardware and system call features are integrated into a unified modality, while a differential analysis algorithm is applied to identify modified memory regions. Then the regions are visualized as images to form another modality. These two inputs are finally fed into a multi-modal model. In their fusion strategy, features with entirely different dimensions and physical meanings are directly flattened and forced into a single-modal branch at the raw-feature stage. This early-stage fusion easily introduces multi-modal noise and dilutes subtle behavioral correlations. Additionally, their method compares the entire memory to generate memory change regions, which is time-consuming and limits its efficiency in practical deployment (Zheng et al. 2024).

Wang et al. proposed HyperMD, a multi-modal detection method leveraging Xen performance counters and runtime memory dumps. However, this approach

necessitates significant modifications to the Xen hypervisor for data collection and relies heavily on prior expert knowledge to define and select effective hardware performance metrics. Regarding fusion mechanism, it adopts a complex joint-driven fusion strategy by designing independent sub-classification losses for individual branches alongside the global fusion loss. This architecture frequently triggers gradient interference during backpropagation (Wang et al. 2025).

Overall, while these studies demonstrate that multi-modal fusion effectively enhances malware detection by integrating complementary information, they still have limitations. Static methods are vulnerable to sophisticated obfuscation, and existing memory-based methods necessitate intrusive modifications to the underlying hypervisor coupled with extensive prior expert knowledge. There remains a critical need for a more effective dynamic multi-modal approach, which can capture both runtime behavioral sequences and memory characteristics without compromising system integrity.

Proposed approach

To improve address the limitations of single-modal data in detecting sophisticated malware, we propose a multi-modal malware detection method. The workflow of our proposed detection method is illustrated in Fig. 1. It consists of three phases: data collection, feature processing, and detection.

In the data collection phase, we execute the collected malware and benign samples in a virtual machine environment. Then, we obtain the memory dump files from outside the VM. At the same time, the API sequences are captured in the sandbox environment and stored as JSON files. In the feature processing phase, the memory dump files are converted into color images, and API sequences are converted to binary features, forming the dataset utilized in this study. After that, we utilize a fine-tuned Residual Neural Network 18 (ResNet18) model to extract image features and a fine-tuned LSTM model to extract text features. Finally, these features are fused and input into a fully connected layer to generate multi-modal features for detection.

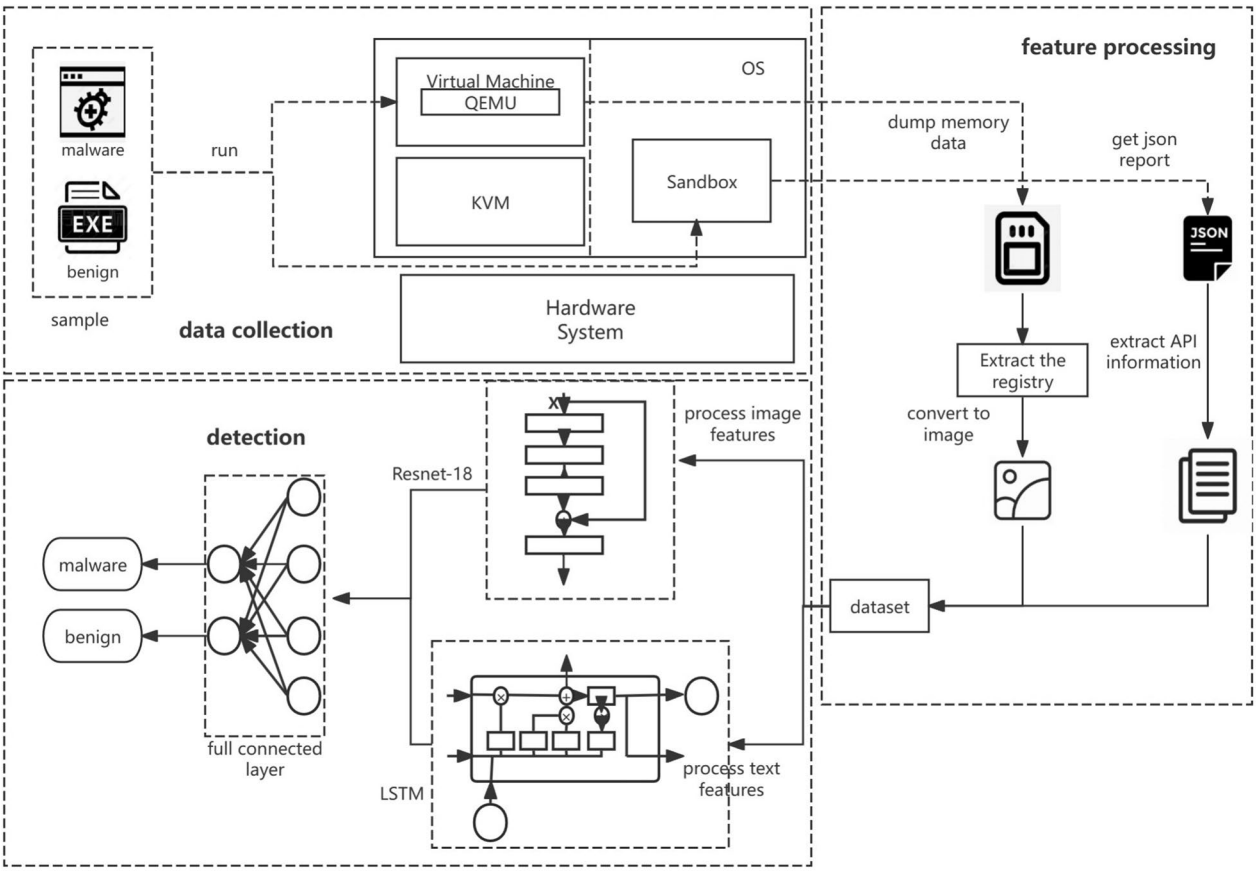


Fig. 1 Workflow of our proposed method

API-based detection

Malware often invokes various APIs during execution to perform malicious activities. Dynamic API-based detection methods therefore execute samples to collect behavioral data. However, running malware directly in a live environment can lead to serious security risks. The sandbox environment provides a controlled setting for collecting detailed behavioral traces. This approach ensures that accurate dynamic behavior data can be obtained without the risk of system compromise.

Therefore, we choose to execute the samples in the Cuckoo sandbox for behavioral data collection. In a sandbox environment, execution behaviors such as API calls, file operations, and network activities, are closely monitored and captured. These captured behaviors are stored in JSON files, which is then parsed to extract specific system activity data relevant to malware detection. The data we extract include API call name, API category, path, registry, string information. Different attributes are processed using feature hashing or statistical measures, such as average string length and character distribution, to generate feature vectors. The extracted features are organized and written into a new binary file for further analysis and classification.

Specifically, to transform heterogeneous behavioral attributes into a uniform numerical representation, we employ a feature hashing mechanism. Categorical attributes are mapped into a fixed-dimensional hash space, producing sparse vector representations of string identifiers. For structural attributes such as file paths and strings, we extract statistical metadata including string length distribution and character entropy to capture latent malicious patterns. Based on these representations, each API call is encoded as a multi-dimensional feature vector, which serves as the basic unit for subsequent sequential modeling.

To analyze API behavior sequences, we chose to leverage an LSTM network (Hochreiter 1997). An LSTM network belongs to the family of RNNs and is capable of effectively processing sequence data. It allows the network to preserve long-term information while processing sequence data, thereby improving its ability to extract sequential features. The model processing workflow is illustrated in Fig. 2.

To standardize input for the LSTM network, we implement a fixed-length sequence truncation and padding strategy. Given that malware execution traces vary significantly in duration, we define a maximum sequence length of 1000. Sequences exceeding this threshold are truncated to capture the initial behaviors, while shorter sequences are zero-padded to maintain dimensional consistency. This approach enables that the model can process the temporal dependencies of API invocations within a uniform tensor framework, facilitating batch training and gradient convergence.

The input feature matrix is first normalized via Batch Normalization to mitigate internal covariate shift. To capture multi-scale local temporal correlations, we process the normalized features through a Gated Convolutional Layer architecture. This consists of three parallel convolutional streams with kernel sizes of 2, 3, and 3, which act as temporal n-gram extractors. The output of each convolution is modulated by a Sigmoid-gated mechanism, allowing the model to adaptively filter informative behavioral motifs while suppressing noise. The resulting gated tensors are concatenated to form a high-dimensional representation that captures diverse behavioral contexts. This fused feature map is then fed into a Bidirectional LSTM (Bi-LSTM), which models the bidirectional temporal dependencies, ensuring that the influence of both preceding and succeeding API calls is preserved in the hidden state representation. Through this multi-layered architecture, the model effectively correlates discrete API attributes with their broader execution context. This model is also integrated into the subsequent multi-modal fusion process.

Memory-based detection

KVM is an open-source virtualization technology integrated into the Linux kernel. Libvirt is an open-source framework for managing virtualization technologies and provides unified access and management capabilities. To address the limitations of in-VM detection, such as the limited perspective, we use out-of-VM detection with the help of libvirt. We constructed a 32-bit Windows 7 virtual machine in a KVM virtualization environment for our experiment. We execute the samples on the VM and capture the memory using the dump command from outside the virtual machine at regular intervals (with a sampling interval of 10–20 s).

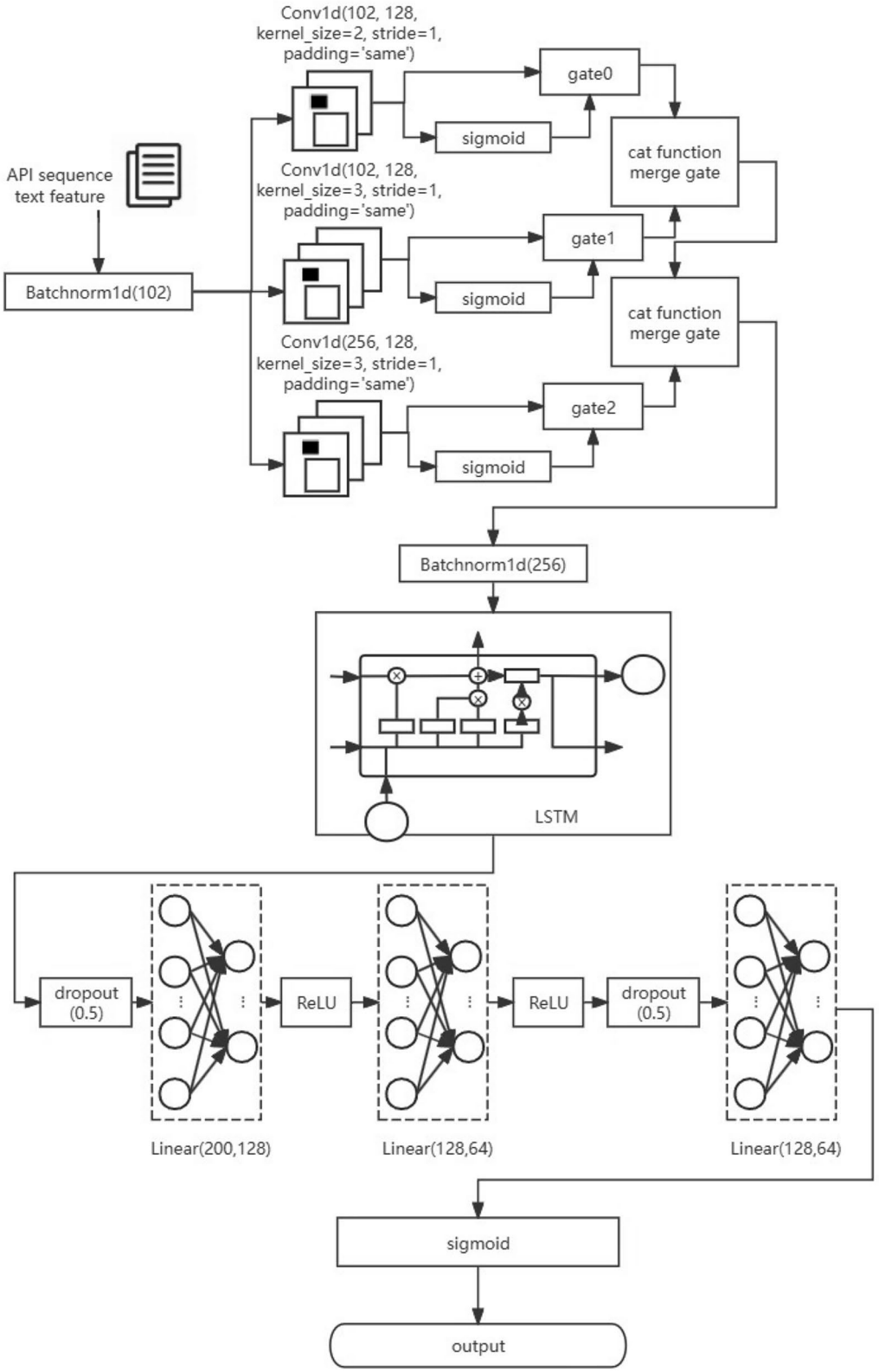


Fig. 2 API-based detection network

Algorithm 1 Extract data and convert to color images

Require: Memory files directory *memory_files_dir*
Require: Output image directory *output_image_dir*
Ensure: PNG images saved in *output_image_dir*
function IMAGE_EXTRACT
  Create *output_image_dir* if it does not exist
  Get list of memory files in *memory_files_dir*
  for all *memory_file* in memory files list **do**
    Extract registry keys using Volatility from *memory_file*
    Parse registry keys to extract offsets
    for all *offset* in *registry_keys* **do**
      Read 98304 bytes of memory data starting at *offset*
      if memory data is valid **then**
        Append memory data to *all_data*
      else
        Skip this *offset*
      end if
    end for
    Concatenate all valid memory data in *all_data*
    Truncate or pad data to $256 \times 256 \times 3$
    Reshape data into $256 \times 256 \times 3$ array
    Normalize pixel values to $[0, 255]$
    Convert data to image and save as PNG in *output_image_dir*
  end for
end function
IMAGE_EXTRACT

Volatility is an open-source tool for memory forensics and analysis, and it is mainly used to analyze computer memory dumps. With Volatility, we can extract Windows registry key-value pairs in memory. The Windows operating system's registry file stores important configuration information, such as user settings. The registry organizes keys, subkeys, values, and related information in a hierarchical tree structure. Helpful information from this structure can be extracted for malware detection. In particular, HKEY_LOCAL_MACHINE stores configuration details for the local computer, while HKEY_CURRENT_ USER contains personal settings for the current user. In the data processing step, we used the Volatility tool to extract the offset positions of HKEY_LOCAL_MACHINE and HKEY_CURRENT_USER.

To reduce resource consumption and simplify the calculation, we process the memory snapshot file into a color image. First, we extract the registry key values and their offsets from the memory file. Then, for each offset, a data block of the specified size is read from the memory dump and checked for validity. After all valid data blocks are merged, the length of the data is adjusted to match the target image size (256×256). The three color channels of each color image contain 196,608 pixel values in total, so we extracted 98,304 pixel values from the offset position of each registry key value for conversion to a color image. Next, the data is reshaped into a 256×256 three-channel array, and the pixel values are normalized to the range of [0, 255]. Finally, a color image is generated and saved as a PNG file. The selection of memory regions is motivated by both system-level importance and security relevance. HKLM stores critical system-wide configurations, including operating system settings, installed software information, driver loading behaviors, and service registrations. These components are closely associated with the execution environment and persistence mechanisms of malware. Different from MDMV(Wang et al. 2024), our method processes smaller selected memory regions. Meanwhile, our subsequent experiments demonstrate that the selected regions are sufficient to achieve strong detection performance. This process is explained in detail in Algorithm 1. The images are shown in Fig. 3.

To analyze the converted memory images, we use a fine-tuned ResNet18 model (He et al. 2016). Its residual block effectively mitigates the problem of vanishing and exploding gradients, helping to support deeper network structures. Meanwhile, ResNet has a flexible structure

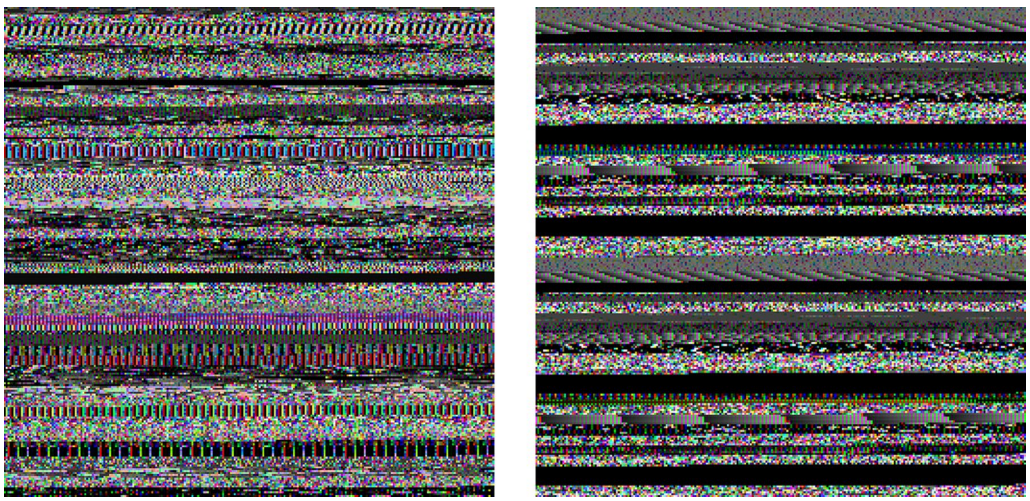


Fig. 3 Comparison of malware and benign generated images. Left: Malware image; Right: Benign image

and is easy to modify. Taking into account these advantages and the limitations of computing resources, ResNet18 was selected as our basic processing model. The images converted from memory are input into the ResNet18 for feature extraction. After being processed by multiple convolutional layers and ReLU activation functions, high-level feature information is extracted. Subsequently, these features are further reduced in dimension through the global average pooling operation to extract global information. The pooled output is flattened into a one-dimensional vector and passed to the fully connected layer to further learn the complex relationships between features. Finally, the output of the fully connected layer is used for classification task to generate the final category label. This structural design aims to fully utilize the feature extraction capabilities of CNN while achieving effective classification through global pooling and fully connected layers. This structure is shown in Fig. 4.

Multi-modal-based detection

In the precedent sections, we present two malware detection methods: one based on API and the other based on an image converted from memory. API-based malware detection is able to capture the malware's runtime behavior. However, benign and malware may exhibit similar API call patterns, increasing the risk of false positives. Moreover, advanced malware may suppress or bypass monitored API calls, delay malicious execution, directly manipulate memory, or detect sandbox environments, resulting in incomplete or misleading API traces. Memory-based malware detection extracts data directly from memory and can capture key characteristics of malware. However, memory images only capture a momentary snapshot, potentially missing transient malicious activities. To combine the strengths of both detection methods, we introduce a multi-modal fusion technique in this section. We adopt a feature-level multi-modal fusion strategy. For each modality, we select a corresponding model for initial training and extract feature vectors.

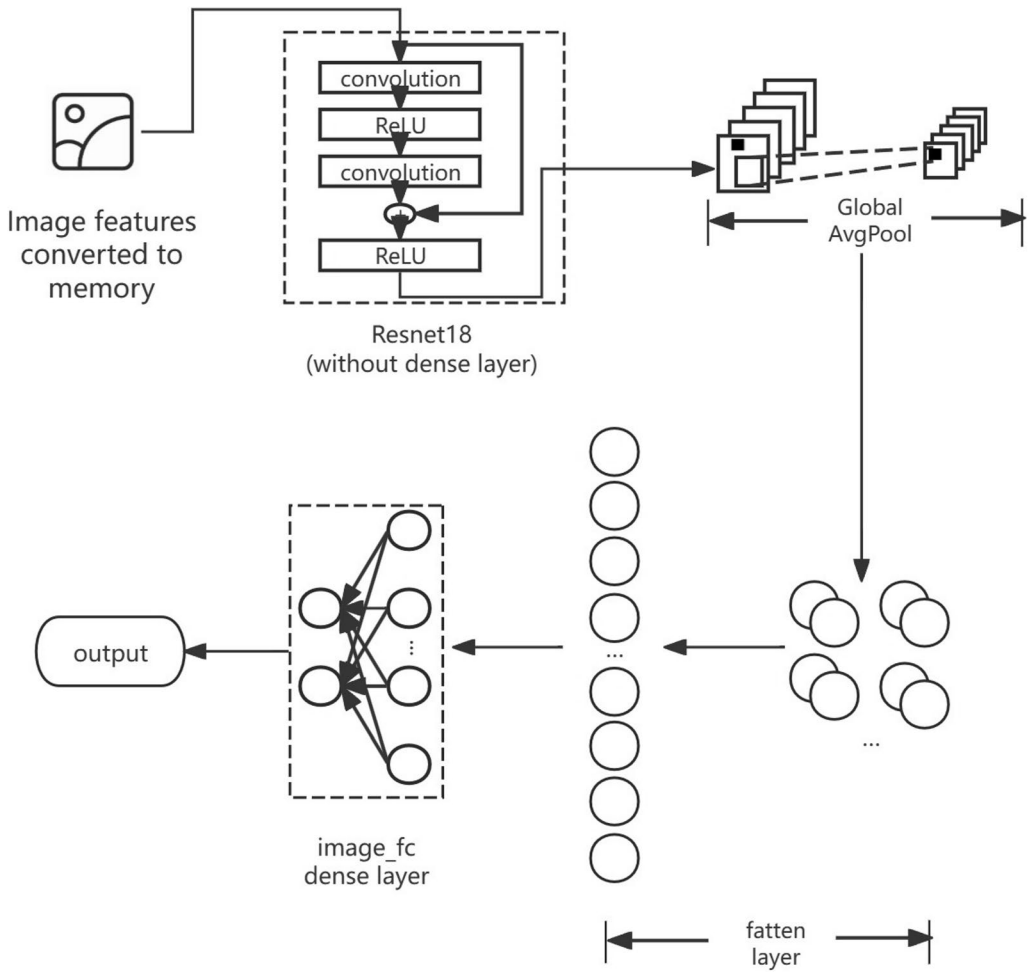


Fig. 4 Memory-based detection network

Then, we integrate these feature vectors for subsequent decision-making. The API sequence captures temporal behavioral information, while the out-of-VM memory feature provides an additional system-state view. As a result, this complementary design improves the robustness of the proposed method against attacks targeting a single observation source.

The structural diagram of the multi-modal model is depicted in Fig. 5. The memory dump image features and API features are processed separately by models, and the resulting outputs are then merged into a composite feature set. This composite set serves as the input to the multi-modal fusion layer. The fused data subsequently passes through a fully connected layer, where the output dimension corresponds to the number of classification categories.

Experiment

Experimental environment and dataset

In this experiment, executable malware samples that can run on a 32-bit Windows 7 system were mainly collected from VirusTotal. After analyzing VirusTotal's report, we chose several ordinary and particularly threatening malware families: CryptoRansomware, a type of encryption ransomware designed to encrypt files on the victim's computer or network, rendering them inaccessible and demanding a ransom; Locker, a variant of ransomware that typically locks the entire system rather than individual files; and Zeus, a Trojan horse with various variants, distributed through multiple channels such as email and malicious websites, which infiltrates the system and monitors user network activities. Additionally, to diversify the dataset and improve the model's ability to detect various types of malware, we selected additional malware variants, such as those employing anti-VM and anti-sandbox techniques. Figure 6 illustrates the composition of the different malware families represented in the dataset. The benign samples in this experiment are mainly from commonly used software and compatible samples selected by the CNET (CNET 2024) website. The entire dataset we selected includes 1570 malware samples and 1557 benign samples. For dataset partitioning, 20% of both malware and benign samples were randomly selected to form the test set, while the remaining 80% were used for model training.

In terms of performance evaluation indicators, we evaluate the method proposed through accuracy, recall, precision, and F1 score. The true positives (TP), false positives (FP), true negatives (TN), and false negatives (FN) are defined in Table 1. Based on these definitions, the evaluation metrics are calculated as follows:

The following metrics are calculated based on the confusion matrix:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{1}$$

$$\text{Recall} = \frac{TP}{TP + FN} \tag{2}$$

$$\text{Precision} = \frac{TP}{TP + FP} \tag{3}$$

$$\text{F1-Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{4}$$

Evaluation of our proposed multi-modal method

In order to verify the effectiveness of our proposed method, we performed a comparative experiment of malware detection based on single image features and API sequence features in the same dataset. The results are shown in Table 2.

The memory-based detection method achieves high accuracy and precision, demonstrating its effectiveness in identifying malicious samples. However, its recall rate is slightly lower, indicating the possibility of missing some malicious samples. In comparison, while the accuracy of the API-based detection method is lower than that of the memory-based approach, it still exhibits strong precision and a competitive F1-score.

Notably, while single-modal methods achieve good performance, multi-modal detection has outperformed them across all metrics. This highlights the effectiveness of the proposed model, which leverages both image and API sequence data to capture distinct dimensions of malware characteristics. Memory images primarily reflect snapshot-based characteristics, while API sequences reveal temporal characteristics. By fusing these two modalities, our proposed model effectively compensates for the limitations inherent in single-modal approaches, thereby significantly enhancing detection accuracy and robustness.

Evaluation of our proposed API-based method

We first evaluate the performance of different API encoding schemes, including one-hot encoding, TF-IDF, and our adopted feature hashing. As shown in Table 3, while TF-IDF and One-hot encoding are widely used in traditional text processing, they exhibit inherent limitations when applied to complex API call sequences for deep learning models.

Specifically, TF-IDF achieves the lowest performance, as it follows a bag-of-words paradigm that emphasizes term frequency while ignoring the sequential order and temporal dependencies of API call sequences. These are

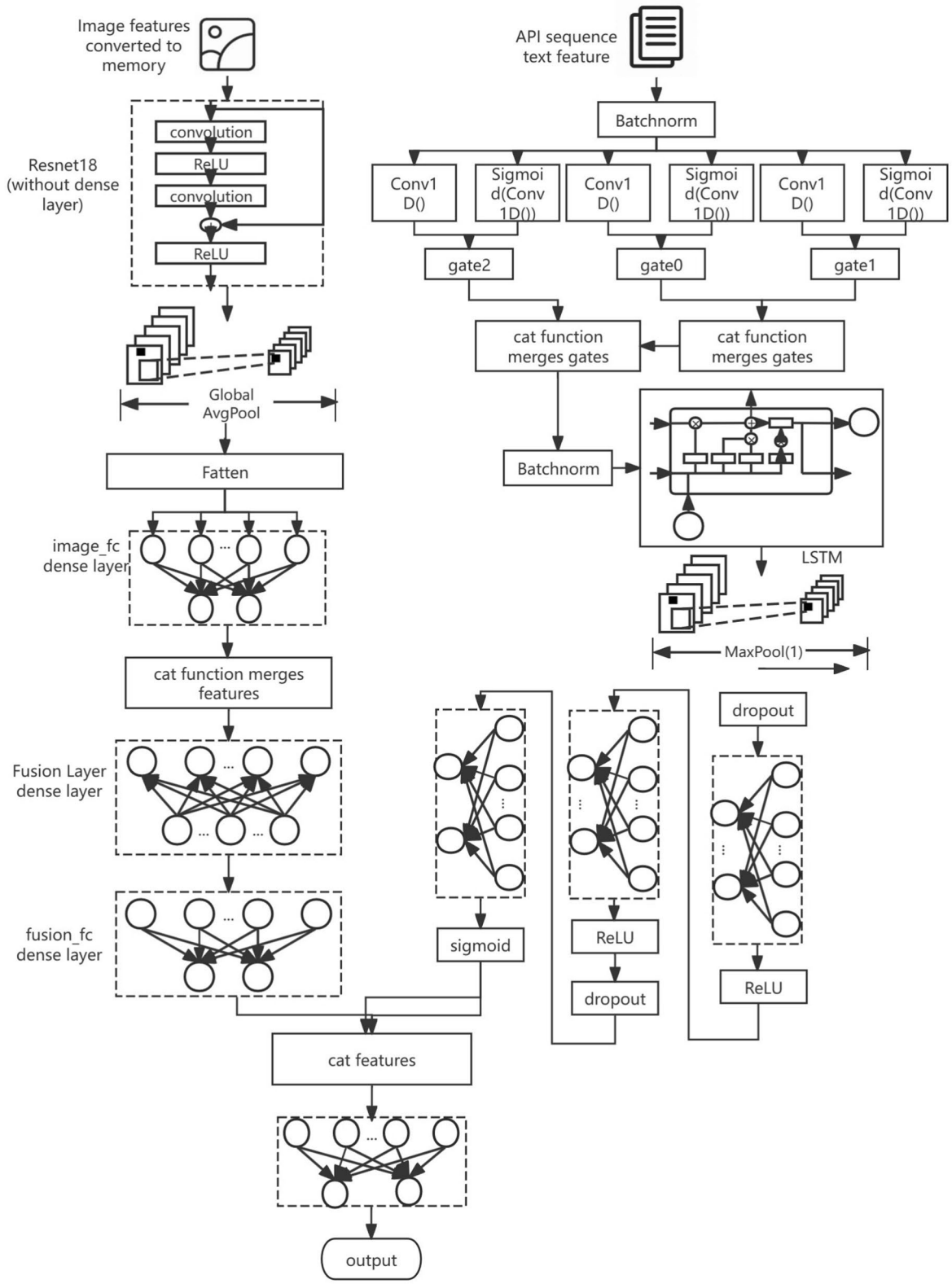


Fig. 5 Multi-modal detection model network

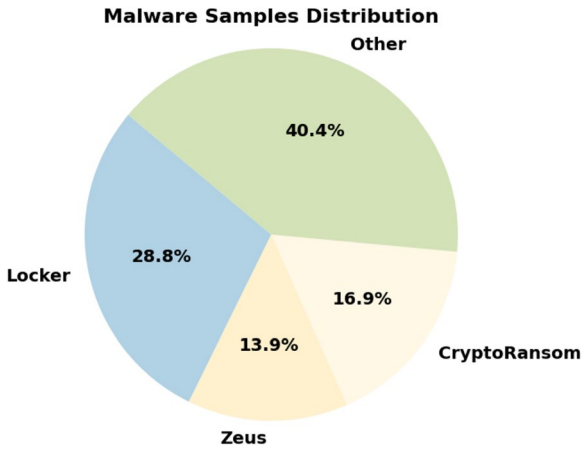


Fig. 6 Experimental sample composition

Table 1 Model evaluation confusion matrix

Actual	Predicted	
	Positive	**Negative**
Positive	TP (True positive)	FN (False negative)
Negative	FP (False positive)	TN (True negative)

Table 2 Comparison of single-modal and multi-modal detections

Method	Accuracy	Precision	Recall	F1-Score
Based on Memory	0.970	0.992	0.948	0.970
Based on API	0.958	0.984	0.933	0.958
Multi-modal	0.986	1.000	0.972	0.986

Table 3 Comparison of different API encoding schemes

Method	Accuracy	Precision	Recall	F1-Score
TF-IDF	0.922	0.924	0.920	0.922
One-hot	0.953	0.945	0.962	0.954
Feature Hashing(Ours)	0.958	0.984	0.933	0.958

vital for LSTM-based behavioral modeling. In contrast, One-hot encoding achieves relatively high recall. However, it results in a high-dimensional and sparse feature space, making it susceptible to the curse of dimensionality as the API vocabulary expands. Moreover, it lacks robustness to out-of-vocabulary (OOV) tokens, limiting its ability to generalize to previously unseen API patterns in evolving malware.

Table 4 Comparison of the other model

Method	Accuracy	Precision	Recall	F1-Score
Transformer (Vaswani et al. 2017)	0.910	0.935	0.878	0.910
Fine-tuned LSTM(Ours)	0.970	0.992	0.948	0.970

Table 5 Comparison of detection accuracy and time cost of different memory extraction methods

Method	Time(s)	Accuracy
Zheng et al. (2024)	1956.2323	0.961
MDMV (Wang et al. 2024)	24.2048	0.968
Our Method	0.0068	0.986

Our feature hashing effectively addresses these challenges by employing a hash function to map high-dimensional API identifiers into a compact, fixed-size vector space. Our approach not only preserves essential behavioral semantics but also demonstrates superior robustness against OOV tokens. Consequently, feature hashing achieves the most balanced performance, particularly reaching an exceptional precision of 0.984. This provides a more discriminative and efficient input for robust malware detection.

Table 4 presents the performance comparison between the fine-tuned LSTM model and the Transformer model for processing the API sequence modality (Vaswani et al. 2017). Although the Transformer architecture has shown remarkable success in various sequence modeling tasks, its performance in our malware classification scenario falls short of that achieved by the fine-tuned LSTM. This discrepancy can be attributed to several factors, including the relatively small size of the training dataset, the moderate length of API sequences, and the strong temporal dependencies intrinsic to API call patterns. Unlike LSTM, which is inherently well-suited for capturing sequential order, the self-attention mechanism in Transformer may dilute strict temporal information. Furthermore, due to its large number of parameters, Transformer is more susceptible to overfitting in data-constrained settings, which in turn impairs its generalization capability.

Evaluation of our proposed memory method

We conduct ablation studies on memory image construction. As shown in Table 5, our method demonstrates a significant advantage in computational efficiency. Zheng et al.'s full memory analysis approach employs a memory difference algorithm to identify the memory change

regions, converting them into RGB images for processing. However, this method requires a full traversal and comparison of several gigabytes of original memory files, leading to extremely high data redundancy. Consequently, feature extraction is highly time-consuming, typically requiring thousands of seconds to complete. The time reported in Table 5 corresponds to the memory feature extraction stage. While all three methods utilize the full memory dump, their subsequent processing strategies differ. Thus, the comparison reflects the time cost introduced by the different memory feature extraction procedures. The end-to-end pipeline also includes sample execution, API collection, memory dumping, model inference, and multi-modal fusion. Among these stages, dynamic sample execution and data acquisition introduce unavoidable runtime overhead. Model inference and feature fusion add relatively limited overhead. The current workflow is therefore suited to sandbox-based and offline malware analysis, where samples can be executed in isolated environments before classification.

In contrast, MDMV's critical memory analysis method reduces data size by selecting the first 16 MB of key regions from memory dump files, and then maps binary sequences into grayscale images using the SimHash algorithm. Nevertheless, this approach still involves complex bitwise operations and feature transformations over relatively large data blocks during SimHash computation, which introduces non-negligible processing overhead.

Additionally, we test various image resolutions (64×64, 128×128, 256×256, 384×384), as shown in Table 6. The experimental results show a substantial performance improvement when increasing the input resolution from 64×64, 128×128 to 256×256. This improvement is primarily attributed to finer granularity at higher resolutions. Specifically, the 256×256 resolution effectively preserves critical spatial hierarchies and micro-patterns in the memory-to-image mapping, which are often degraded by quantization noise and aliasing effects in lower resolutions. Although higher resolution theoretically contains more details, the model did not achieve superior performance with a 384×384 input. This phenomenon may be attributed to the increased sparsity of the feature matrix at higher dimensions, or potentially a slight overfitting of the model to the training samples in the high-resolution feature space, which consequently leads to a minor reduction in generalization capability. In conclusion, the 256×256 resolution provides the best balance between feature richness and model generalization.

To gain deeper insight into the contribution of each modality, we also evaluate the memory-based and API-based models independently by comparing them against other advanced methods. Table 7 presents the comparison between our proposed model and the VGG16 (Visual Geometry Group 16-layer network) network (Simonyan and Zisserman 2014). A recall of 1.0 indicates that the model predicts all samples as positive. However, both the accuracy and precision performance are poor, with the results only slightly higher than 0.5. This result often indicates a severe issue of overfitting or an inability to adapt to class imbalance in binary classification tasks. This may be due to the excessive number of parameters in the VGG16 network, which is prone to overfitting. In addition, VGG16 has no residual structure, and deep information can be degraded. Moreover, our images are pseudo-images converted from memory dump data, which do not have clear spatial structures like natural images. The strength of VGG16 is to recognize local patterns (such as edges, textures) in natural images, while in memory transfer images, these patterns may not have obvious semantics.

Evaluation of different fusion strategies

To determine the optimal integration mechanism, we evaluate three fusion strategies: Decision-level Fusion (averaging prediction probabilities), Cross-modal Attention Fusion (dynamic weighting), and Feature-level Concatenation (our proposed method). As summarized in Table 8, the Feature-level Concatenation strategy

Table 6 Comparison of different image sizes

Method	Accuracy	Precision	Recall	F1-Score
64×64	0.850	0.972	0.722	0.828
128×128	0.896	0.839	0.978	0.903
256×256	**0.970**	**0.992**	**0.948**	**0.970**
384×384	0.948	0.983	0.912	0.947

Table 7 Comparison of the other model

Method	Accuracy	Precision	Recall	F1-Score
VGG16 (Simonyan and Zisserman 2014)	0.503	0.503	1.000	0.669
Fine-tuned ResNet18 (Ours)	0.958	0.984	0.933	0.958

Table 8 Comparison of different fusion strategies

Method	Accuracy	Precision	Recall	F1-Score
Decision-level	0.970	1.000	0.939	0.970
Cross-modal attention	0.505	0.500	0.796	0.614
Feature-level (Ours)	0.986	1.000	0.972	0.986

achieves the highest detection accuracy. This suggests that by concatenating deep features before the final classification layers, the model can effectively learn the complex and non-linear interdependencies between temporal API behaviors and spatial memory patterns through the subsequent fully connected layers.

In contrast, Decision-level Fusion yields inferior results as it isolates the learning processes of each modality, preventing the model from capturing fine-grained correlations until the final prediction stage. Furthermore, although Cross-modal Attention is widely adopted in multi-modal learning, it performs suboptimally in our framework. It achieves near-random precision, suggesting a bias toward over-predicting the positive class. We attribute this behavior to the structural heterogeneity between the two modalities. First, 1D API sequences and 2D memory dumps characterize software behavior through fundamentally distinct modalities. While the former captures the discrete, sequential logic of execution, the latter provides a flattened, spatial snapshot of the underlying raw data. This intrinsic cross-modal nature results in a significant mismatch in both abstraction levels and semantic granularity. Second, this substantial semantic gap poses formidable challenges for cross-modal learning of shared representations. This makes it difficult to learn shared representations and establish of robust correspondences between different modalities.

Feature-level Concatenation provides a simpler but more effective fusion strategy, enabling the model to learn complementary representations without introducing additional alignment complexity. This leads to more stable optimization and superior overall performance.

Comparison with other methods

Comparison with other single-modal methods

First, we evaluate our individual detection branches against specialized single-modal methods. The detection results are presented in Table 9. To ensure a fair comparison, all baselines were re-implemented on the same dataset, with method-specific features extracted from the same raw malware and benign samples.

In the domain of static image-based detection, we select IMCFN for comparison (Vasan et al. 2020). IMCFN transforms the original malware binary files into color images. After obtaining the image features, it employs a fine-tuned CNN model for malware detection. While IMCFN leverages image features effectively, it relies solely on static feature representations. That makes it easily bypassed by obfuscation and fail to capture the dynamic behavioral patterns exhibited by malware. Although it performs well, it struggles with sophisticated and evolving malware samples. Our approach addresses this limitation and significantly enhances the detection performance.

Table 9 Comparison with other single-modal methods

Method	Accuracy	Precision	Recall	F1-Score
IMFCN (Vasan et al. 2020)	0.943	0.948	0.943	0.942
Li et al. (2022)	0.947	0.952	0.937	0.944
MDMV (Wang et al. 2024)	0.950	0.966	0.933	0.949
Based on API (Ours)	0.958	0.984	0.933	0.958
Based on Memory (Ours)	0.970	0.992	0.948	0.970

Regarding API-based detection, we compare our approach with a novel malware detection framework proposed by Li et al. (2022). This method utilizes deep learning models to capture and integrate more meaningful features of API sequences. While their method successfully captures certain intrinsic features, our utilized model achieves superior results in single API-based modality detection. This validates the effectiveness of our approach. Moreover, Li et al.'s method relies on a single-modal approach, which limits its effectiveness in handling complex behaviors. By incorporating image data, our approach expands the feature representation, significantly enhancing the model's robustness and detection accuracy.

Furthermore, in the domain of memory-to-image detection, we evaluate our method against MDMV (Wang et al. 2024), which focuses on extracting key memory regions and generating grayscale images by the SimHash algorithm. As shown in Table 9, our method outperforms it across multiple evaluation metrics. This performance gap suggests that MDMV's reliance on fixed-size key regions and SimHash-based mapping may lead to the loss of fine-grained behavioral information. In contrast, our method extracts more precise memory segments and utilizes a higher resolution of 256×256 images, which better preserves discriminative spatial patterns. By retaining richer structural information from memory data, our utilized model demonstrates superior robustness and a lower false-positive rate compared to the MDMV framework.

Comparison with other multi-modal methods

To further evaluate the effectiveness of the proposed approach, we compare our method with several state-of-the-art multi-modal frameworks, including HYDRA (static feature-based), Zheng et al. (behavioral sequence-based), and HyperMD (performance counter-based). The results of the comparison are shown in Table 10. To ensure an equitable comparison, all baseline methods were re-evaluated using the same dataset, utilizing the

Table 10 Comparison of different multi-modal Methods

Method	Accuracy	Precision	Recall	F1-Score
HYDRA (Gibert et al. 2020)	0.896	0.939	0.831	0.882
Zheng et al. (2024)	0.957	0.964	0.948	0.956
HyperMD (Wang et al. 2025)	0.976	0.963	0.995	0.978
Multi-modal (Ours)	0.986	1.000	0.972	0.986

specific input features extracted from the same original malware and benign datasets.

HYDRA integrates various static modalities, including byte sequences, mnemonic sequences, and Windows API function calls, using a multi-modal deep learning architecture. By combining hand-engineered features with end-to-end learning components, it demonstrates robust performance in general static analysis. However, its reliance on static features makes it inherently vulnerable to sophisticated malware. As a result, HYDRA may fail to capture the dynamic execution features required for detecting sophisticated and evasive threats.

Zheng et al. propose a method that performs dimensionality reduction on the entire VM storage file to construct global feature representations. While this method provides entire view of memory content, it can easily introduce noise interference from similar or redundant regions in memory, thereby negatively affecting detection performance. Moreover, as discussed earlier, processing the entire VM's memory file will bring enormous system computational overhead and storage pressure, making it less suitable for efficient detection.

HyperMD achieves high detection accuracy by leveraging low-level Xen performance counter data and target memory feature images, which captures fine-grained execution behaviors. However, its effectiveness heavily depends on complex semantic reconstruction of low-level signals, which requires substantial prior knowledge and careful feature engineering. In addition, its reliance on hypervisor modifications restricts deployment flexibility and cross-platform applicability.

In contrast, our proposed multi-modal method achieves better detection accuracy. Unlike purely static approaches, our method captures both behavioral and structural features of malware, improving the ability to detect sophisticated malware. Compared to whole-memory processing strategies, our selective memory feature extraction reduces noise while preserving critical information, thereby achieving a better balance between detection accuracy and computational efficiency. Additionally, our framework does not rely on intrusive system modifications or prior knowledge, making it more practical for real-world deployment. By leveraging non-intrusive dynamic analysis and optimized feature fusion, our approach outperforms HYDRA, Zheng et al., and HyperMD across multiple evaluation metrics. The dataset is mainly collected from VirusTotal and CNET. In practical deployment, previously unseen malware or samples from different sources may differ from the training data in both API behavior and memory characteristics. As a result, the detection performance may vary under such distribution shifts. Further evaluation on more diverse data is therefore needed to assess the generalization of the proposed method in practical environments.

Overall, the above results consistently demonstrate the effectiveness of our method across different detection settings. Compared against the static image-based, API-based, and entire memory-based malware detection methods, our proposed method demonstrates consistently better performance across both single-modal and multi-modal evaluations. The single-modal comparisons indicate that while other methods perform well in their specific areas, they fail to effectively integrate multi-dimensional features, resulting in suboptimal performance in sophisticated malware detection. Building upon these observations, the multi-modal comparison further highlights that simply relying on a single type of feature is insufficient for robust detection. Our multi-modal approach overcomes the limitations of single-modal methods by combining the API sequences with the image features converted from memory files. This integration not only improves accuracy but also provides a richer feature representation by combining complementary behavioral and memory information.

Conclusion and future work

We propose a malware detection method based on multi-modal fusion. This method extracts the image features from the memory dump files out-of-VM and the text features of API sequences obtained in-VM. These features are respectively processed through fine-tuned CNN and LSTM networks. The outputs of these networks are combined to generate multi-modal features for enhanced malware detection. The fusion of multi-modal information highlights its adaptability and flexibility in addressing evolving malware challenges. It shows the significant potential of multi-modal approaches in malware detection.

In the future, we aim to explore additional feature extraction techniques and further optimize the deep learning models. We will also evaluate the proposed method on newly emerging malware families and samples from different sources. This will help assess its generalization under practical distribution shifts. Moreover, the end-to-end runtime is affected by dynamic sample

execution and data acquisition. We plan to optimize these stages while maintaining good performance.

Acknowledgements
Not applicable.

Author contributions
Lexin Jia: original draft preparation, formal analysis Jian Zhang: review & editing, formal analysis Xiangyi Wang: software analysis, data collection Wei Zheng: review & editing

Funding
This work was supported by the National Key R&D Program of China (2022YFB3103202) and the National Science and Technology Major Project of China (2025ZD1501602).

Data availability
The datasets used and/or analyzed during the current study are available from the corresponding author on reasonable request.

Declarations

Ethics approval and consent to participate
Not applicable.

Consent for publication
Not applicable.

Competing interests
The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Received: 12 November 2025 Accepted: 9 October 2026
Published online: 23 October 2026

References

Aboaoja FA, Zainal A, Ghaleb FA, Abd Arrain M, Abuarqoub A (2022) Malware detection issues, challenges, and future directions: A survey. Appl Sci 12(17):8482
Almashhadani AO, Carlin D, Kaiiali M, Sezer S (2022) MFMCNS: A multi-feature and multi-classifier network-based system for ransomworm detection. Comput Secur 121:102860
Amer E, Zelinka I (2020) A dynamic Windows malware detection and prediction method based on contextual understanding of API call sequence. Comput Secur 92:101760
Arfeen A, Khan MA, Zafar O, Ahsan U (2022) Process based volatile memory forensics for ransomware detection. Concurr Comput Practice Exp 34(4):6672
AV-TEST Institute: Malware Statistics & Trends 2024. (2024). https://www.av-test.org/en/statistics/malware/ Accessed 2025-11-12
Bensaoud A, Kalita J (2024) CNN-LSTM and transfer learning models for malware classification based on opcodes and API calls. Knowl-Based Syst 290:111543
Brosolo M, Asmitha KA, Conti A, Rafidha RKA, Shafi MKP, Nicolazzo S, Nocera A, Vinod P (2026) Security through the eyes of ai: How visualization is shaping malware detection. Comput Sci Rev 61:100914
Carrier T (2021) Detecting obfuscated malware using memory feature engineering. PhD thesis, University of New Brunswick, Fredericton, NB, Canada
Chen S, Lang B, Liu H, Chen Y, Song Y (2024) Android malware detection method based on graph attention networks and deep fusion of multimodal features. Expert Syst Appl 237:121617
CNET: Download Software for Windows. Accessed: 2024-01-14 (2024). https://download.cnet.com/windows/28/
Cui Y, Sun Y, Lin Z (2023) Droidhook: a novel api-hook based android malware dynamic analysis sandbox. Autom Softw Eng 30(1):10
Cui L, Yin J, Cui J, Ji Y, Liu P, Hao Z, Yun X (2024) Api2vec++: Boosting api sequence representation for malware detection and classification. IEEE Trans Software Eng 50(8):2142–2162
Darem A, Abawajy J, Makkar A, Alhashmi A, Alanazi S (2021) Visualization and deep-learning-based malware variant detection using OpCode-level features. Futur Gener Comput Syst 125:314–323
Dehfouli Y, Lashkari AH (2025) Memory analysis for malware detection: A comprehensive survey using the oscar methodology. ACM Comput Surv 58(4):1–58
Dener M, Ok G, Orman A (2022) Malware detection using memory analysis data in big data environment. Appl Sci 12(17):8604
Gbedawo VW, Agyeman GO, Ankah CK, Daabo MI (2023) An overview of computer memory systems and emerging trends. Am J Electr Comput Eng 7(2):19–26
Gibert D, Mateu C, Planes J (2020) The rise of machine learning for detection and classification of malware: Research developments, trends and challenges. J Netw Comput Appl 153:102526
Gibert D, Mateu C, Planes J (2020) Hydra: A multimodal deep learning framework for malware classification. Comput Secur 95:101873
He X, Li R (2024) Malware detection for container runtime based on virtual machine introspection. J Supercomput 80(6):7245–7268
He K, Zhang X, Ren S, Sun J (2016) Deep residual learning for image recognition. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2016). IEEE, Las Vegas, NV, USA, pp 770–778
Hochreiter S (1997) Long Short-term Memory. Neural Computation MIT-Press
International Organization for Standardization: ISO/IEC 27032:2023 — Information Security, Cybersecurity, and Privacy Protection — Guidelines for Cybersecurity. (2023). https://www.iso.org/obp/ui/#iso:std:iso-iec:27032:ed-2:v1:en:en%20ISO/IEC%2027032:2023(en)
Jeon J, Jeong B, Baek S, Jeong Y-S (2024) Static multi feature-based malware detection using multi spp-net in smart iot environments. IEEE Trans Inf Forensics Secur 19:2487–2500
Karat G, Kannimoola JM, Nair N, Gopalakrishnan S (2024) CNN-LSTM hybrid model for enhanced malware analysis and detection. Procedia Comput Sci 233:492–503
Kim, D, Majlesi-Kupaei, A, Roy J, Anand K, ElWazeer K, Buettner D, Barua R (2017) Dynodet: Detecting dynamic obfuscation in malware. In: Proceedings of the 14th International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA), Bonn, Germany, pp. 97–118. Springer
Kim TG, Kang B, Rho M, Sezer S, Im EG (2018) A multimodal deep learning method for android malware detection using various features. IEEE Trans Inf Forensics Secur 14(3):773–788
Li C, Lv Q, Li N, Wang Y, Sun D, Qiao Y (2022) A novel deep framework for dynamic malware detection based on API sequence intrinsic features. Comput Secur 116:102686
Li S, Li Y, Wu X, Al Otaibi S, Tian Z (2022) Imbalanced malware family classification using multimodal fusion and weight self-learning. IEEE Trans Intell Transp Syst 24(7):7642–7652
Liu K, Li Y, Xu N, Natarajan P (2018) Learn to combine modalities in multimodal deep learning. arXiv preprint arXiv:1805.11730
Lu X, Jiang F, Zhou X, Yi S, Sha J, Lio P (2019) ASSCA: API sequence and statistics features combined architecture for malware detection. Comput Netw 157:99–111
Maniriho P, Mahmood AN, Chowdhury MJM (2024) MeMalDet: A memory analysis-based malware detection framework using deep autoencoders and stacked ensemble under temporal evaluations. Comput Secur 142:103864
Mishra P, Aggarwal P, Vidyarthi A, Singh P, Khan B, Alhelou HH, Siano P (2021) VMShield: Memory introspection-based malware detection to secure cloud-based services against stealthy attacks. IEEE Trans Industr Inf 17(10):6754–6764
Or-Meir O, Nissim N, Elovici Y, Rokach L (2019) Dynamic malware analysis in the modern era–A state of the art survey. ACM Comput Surveys (CSUR) 52(5):1–48
Patil R, Dudeja H, Modi C (2020) Designing in-VM-assisted lightweight agent-based malware detection framework for securing virtual machines in cloud computing. Int J Inf Secur 19(2):147–162

Qin T, Peng B, Lei J, Yao Y, Huang Q (2025) Modeling intra-and inter-modal correlations for incomplete multi-modal 3d shape clustering. IEEE Trans Multimedia
Rozehkhani SM, Mahan F, Pedrycz W (2024) Efficient cloud data center: An adaptive framework for dynamic virtual machine consolidation. J Netw Comput Appl 226:103885
Shree R, Shukla AK, Pandey RP, Shukla V, Bajpai D (2022) Memory forensic: Acquisition and analysis mechanism for operating systems. Mater Today Proc 51:254–260
Sihwail R, Omar K, Ariffin KAZ (2018) A survey on malware analysis techniques: Static, dynamic, hybrid and memory analysis. Int J Adv Sci Eng Inf Technol 8(4–2):1662–1671
Simonyan K, Zisserman A (2014) Very deep convolutional networks for large-scale image recognition. arXiv preprint arXiv:1409.1556
Syeda DZ, Asghar MN (2024) Dynamic malware classification and API categorisation of windows portable executable files using machine learning. Appl Sci 14(3):1015
Tobiyama S, Yamaguchi Y, Shimada H, Ikuse T, Yagi T (2016) Malware detection with deep neural network using process behavior. Proceedings of the IEEE 40th Annual Computer Software and Applications Conference (COMPSAC), vol 2. IEEE, Atlanta, GA, USA, pp 577–582
Vasan D, Alazab M, Wassan S, Naeem H, Safaei B, Zheng Q (2020) IMCFN: Image-based malware classification using fine-tuned convolutional neural network architecture. Comput Netw 171:107138
Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones, L, Gomez AN, Kaiser Ł, Polosukhin I (2017) Attention is all you need. Advances in neural information processing systems. 30
Wang X, Zhang J, Jia L, Meng Z, Xing L (2024) Mdmv: A malware detection method based on memory and visualization on kvm. International Conference on Mobile and Ubiquitous Systems: Computing, Networking, and Services. Springer, pp 570–590
Wang X, Zhang J, Xing L, Yang M, Meng Z, Jia L (2025) Hypermd: A multi-modal malware detection method using performance counters and process memory on xen platform. IEEE Trans Inf Forensics Secur
Wei X, Li C, Lv Q, Li N, Sun D, Wang Y (2026) Mitigating the impact of malware evolution on api sequence-based windows malware detectors. IEEE Trans Inf Forensics Secur 21:45–60
Wu P, Gao M, Sun F, Wang X, Pan L (2025) Multi-perspective api call sequence behavior analysis and fusion for malware classification. Comput Secur 148:104177
Yamany B, Elsayed MS, Jurcut AD, Azer MA (2024) A Holistic approach to ransomware classification: leveraging static and dynamic analysis with visualization. Information 15(1):46
Yi X, Liu Y, Yang B, Zhang J (2025) Federated continual learning via orchestrating multi-scale expertise. The Thirty-ninth Annual Conference on Neural Information Processing Systems.
Yücel Ç, Koltuksuz A (2020) Imaging and evaluating the memory access for malware. Forensic Sci Int Digital Invest 32:200903
Zhang X, Zhang M, Zhang Y, Zhong M, Zhang X, Cao Y, Yang M (2022) Slowing down the aging of learning-based malware detectors with api knowledge. IEEE Trans Dependable Secure Comput 20(2):902–916
Zhang S, Hu C, Wang L, Mihaljevic MJ, Xu S, Lan T (2023) A malware detection approach based on deep learning and memory forensics. Symmetry 15(3):758
Zhang W, Shi H, Zhao Y, Feng Z, Lovreglio R (2024) MMAF-Net: Multi-view multi-stage adaptive fusion for multi-sensor 3D object detection. Expert Syst Appl 242:122716
Zhang S, Gao M, Wang L, Xu S, Shao W, Kuang R (2025) A malware-detection method using deep learning to fully extract api sequence features. Electronics 14:167
Zheng L (2022) Zhang J (2022) A new malware detection method based on vmcadr in cloud environments. Secur Commun Netw 1:4208066
Zheng L, Zhang J, Lin F, Wang X (2023) Feature-fusion-based abnormal-behavior-detection method in virtualization environment. Electronics 12(16):3386
Zheng L, Zhang J, Wang X, Lin F, Meng Z (2024) Multimodal-based abnormal behavior detection method in virtualization environment. Comput Secur 143:103908

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.